

Problem A. count

Time Limit 5000 ms

Mem Limit 524288 kB

如果一个序列满足序列长度为 n ，序列中的每个数都是1到 m 内的整数，且所有1到 m 内的整数都在序列中出现过，则称这是一个挺好序列。

对于一个序列 A ，记 $f_A(l, r)$ 为 A 的第 l 个到第 r 个数中最大值的下标（如果有多个最大值，取下标最小的）。

两个序列 A 和 B 同构，当且仅当 A 和 B 长度相等，且对于任意 $i \leq j$ ，均有 $f_A(i, j) = f_B(i, j)$ 。

给出 n, m ，求有多少种不同构的挺好序列。答案对998244353取模。

输入格式

一行两个正整数 n, m 。

输出格式

一行一个整数，表示有多少种不同构的挺好序列。

样例一

input

3 2

output

4

explanation

一共6种挺好序列：2 1 1，1 2 1，1 1 2，1 2 2，2 1 2，2 2 1。其中2 1 1和2 2 1同构，1 2 1和1 2 2同构，所以一共有4种不同构的挺好序列。

样例二

input

8 5

output

1341

样例三

input

100000 99999

output

944488805

样例四

input

300 200

output

430256456

样例五

input

2000 1000

output

267823945

样例六

input

100000 50000

output

779381353

限制与约定

子任务	$n, m \leq$	特殊性质	分值
1	8		10
2	100000	$m \geq n - 1$	10
3	300		30
4	2000		20
5	100000		30

时间限制：5s

空间限制：512MB

下载

[样例数据下载](#)

Problem B. 地铁规划

Time Limit 3000 ms

Mem Limit 524288 kB

这是一道交互题。

新首都跳蚤利亚需要建立地铁线路！hehe 蚤负责了这个项目。

跳蚤利亚有 n 个地铁站，还有 m 条线路计划设立，第 i 条铁轨将在 u_i 和 v_i 之间建立一条双向线路（ $u_i \neq v_i$ ）。可能有两条线路连接的地铁站相同。

由于跳蚤利亚是面向未来的节能环保城市，hehe 蚤希望地铁的建设也要避免浪费。hehe 蚤仔细地审查工程建设方案，并定义了什么叫做“绿色区间”：对于任何满足 $1 \leq l \leq r \leq m$ 的整数 l, r ，我们称 $[l, r]$ 是一个区间；如果在只修建所有 $u_i, v_i \in [l, r]$ 的线路的情况下，地铁网络不包含环，那么我们称这个区间是绿色的。

hehe 蚤突然好奇：有多少个区间 $[l, r]$ 是绿色的呢？但这样的好奇持续了 10^{-9} 秒后，hehe 蚤就想到了做法。如果一个区间是绿色的，那它的子区间也是绿色的。于是可以用双指针，对于所有的区间左端点 $l \in [1, m]$ ，计算出使得 $[l, r]$ 是绿色的最大的右端点值 r 即可。而维护是否有环这一信息，只需使用跳蚤国最新自研数据结构 hehe 树。

作为跳蚤国顶级科学家之一，hehe 蚤刚在大量系统中植入了 hehe 树，所以这问题已经变得没意思了。为了挑战自我，hehe 蚤拿出了一台植入 hehe 树失败的机器，仅包含部分 hehe 树功能。具体来说，这台机器支持：

1. 给定 id ，在图中建立双向线路 u_{id}, v_{id} ，注意，如果假如建立后图中产生了环，机器会报错。
2. 拆除图中加入时间最晚的一条双向线路。
3. 给定 id ，询问 u_{id}, v_{id} 两点间是否可以通过已经建立的双向线路连通。

拿出了这台机器后，hehe 蚤又得出了一个基于分治的强大的做法，并给出了答案。hehe 蚤大呼简单。

但 hehe 蚤又想了想，觉得自己应该在跳蚤国内弘扬这股“麻烦自己，麻烦别人”的挑战精神。于是 hehe 蚤把这台机器交给了你。他会从小到大依次对每个左端点 $l \in [1, m]$ ，向你询问，所有以 l 为左端点的绿色区间 $[l, r]$ 中最大的右端点值 r 。因为 hehe 蚤觉得这个问题太简单，所以在你计算答案 r 时，你不可以让机器加入编号大于 r 的线路。

一句话题意：有一个长度为 m 的边序列，你需要使用交互库提供的可撤销并查集接口，在线地对于每个 l 求出最大的 r ，使得区间 $[l, r]$ 内的边不形成环。

任务

本题仅支持 C++。

你必须引用 `subway.h` 头文件。

你需要实现下面的过程：

```
1 | void init(int n, int m, int lim);
```

这个过程将在所有 `solve` 调用前，恰好被调用一次。

其中 n 表示地铁站数量， m 表示线路数量， lim 表示你可以使用 `merge` 次数的上限。

```
int solve(int l);
```

这个过程将被调用恰好 m 次，且第 i 次调用的 `l` 一定是 i 。你需要返回 r_l 表示最大的 r 满足区间 $[l, r]$ 中的边不成环。

你可以在 `solve` 的过程中调用以下过程和交互库交互：

```
void merge(int x);
```

这个函数的作用是建立双向线路 u_x, v_x 。

这个函数只能在 `solve` 过程中被调用，并且必须满足 $x \in [l, r_l]$ ，其中 l 是当前 `solve` 过程的参数，以及加入这条边不会使 hehe 树维护的图成环，否则你的交互过程会被判定为失败。

你调用这个函数的次数不能超过 lim 次，并且如果你调用这个函数的次数超过了 2×10^7 ，你的交互过程将被判定为不合法。

```
void undo();
```

这个函数将撤销最后一次没有被撤销的 `merge` 操作，如果没有可以被撤销的 `merge` 操作，你的交互过程会被判定为失败。

```
bool check(int x);
```

如果 u_x 和 v_x 联通，函数将返回 `false`，否则返回 `true`。

换言之，返回值表示加入第 x 条线路是否不会成环。

不同 `solve` 轮次间 hehe 蚤维护的图将被保留，并且初始不包含任何边。

评测方式

样例评测库将读入如下格式的输入数据：

第一行包括三个正整数 n, m 和 lim ，分别表示地铁站个数，线路条数和交互限制。

接下来 m 行，每行两个整数 u_i, v_i ，表示一条线路。

样例评测库将输出如下格式的输出数据：

对于每一组数据，如果你交互过程合法，交互库将输出 `Success` 或 `Too many merge`，并在第二行输出你调用 `merge` 的次数，第三行输出你求出的答案，否则会输出 `Wrong Answer [id]`，你可以查阅样例评测库来确定错误类型。

[交互式类型的题目怎么本地测试](#)

最终测试中，线路序列是确定的，不会因为你的询问而改变。

样例

input

4 4 1000
1 2
2 3
3 4
4 2

output

Success
Count of merge: 6
3 3 4 4

explanation

以下是一个合法的交互过程。

交互库调用	选手调用	返回	并查集状态
<code>init(4, 4, 1000)</code>			(1), (2), (3), (4)
<code>solve(1)</code>			(1), (2), (3), (4)
	<code>merge(1)</code>		(1, 2), (3), (4)
	<code>check(2)</code>	<code>true</code>	(1, 2), (3), (4)
	<code>merge(2)</code>		(1, 2, 3), (4)

交互库调用	选手调用	返回	并查集状态
	check(3)	true	(1, 2, 3), (4)
	merge(3)		(1, 2, 3, 4)
	check(4)	false	(1, 2, 3, 4)
solve		3	(1, 2, 3, 4)
solve(2)			(1, 2, 3, 4)
	undo()		(1, 2, 3), (4)
	undo()		(1, 2), (3), (4)
	undo()		(1), (2), (3), (4)
	merge(3)		(1), (2), (3, 4)
	merge(2)		(1), (2, 3, 4)
	check(4)	false	(1), (2, 3, 4)
solve		3	(1), (2, 3, 4)
solve(3)			(1), (2, 3, 4)
	undo()		(1), (2), (3, 4)
	check(4)	false	(1), (2), (3, 4)
	merge(4)		(1), (2, 3, 4)
solve		4	(1), (2, 3, 4)
solve(4)			(1), (2, 3, 4)
solve		4	(1), (2, 3, 4)

其中 `solve` 的返回值均在若干行后的，标注着交互库调用 `solve` 的行。

数据范围与提示

保证若你的交互过程满足题目限制，且 `check` 次数不超过 8×10^6 ，交互库不会占用超过 1s 时间和 32MB 空间。

子任务编号	$n \leq$	$m \leq$	$lim \geq$	特殊性质	分值
1	1000	1000	10^6	无	10
2	10^4	2×10^4	5×10^6	无	30
3	10^5	2×10^5	5×10^6	数据随机	30
4	10^5	2×10^5	5×10^6	无	30

对于所有数据， $2 \leq n, m, lim \leq 5 \times 10^6$ 。

时间限制：3s

空间限制：512MB

Problem C. Fake Plastic Trees 2

Statement [PDF](#)

English

You are given a tree with N vertices numbered from 1 to N . The tree is vertex-weighted. In other words, each vertex of the tree is assigned a nonnegative integer weight.

We will delete some edges from the tree. After the deletion, for each connected component, the sum of vertex weights should be in the range $[L, R]$.

For all integers $0 \leq i \leq K$, determine if we can achieve this goal by deleting exactly i edges.

Input

The first line contains a single integer T , the number of test cases. Then T test cases follow, each following the given specification:

The first line of each test case contains four integers N, K, L , and R ($1 \leq N \leq 1000, 0 \leq K \leq \min(50, N - 1), 0 \leq L \leq R \leq 10^{18}$).

The next line contains N integers $A_1, A_2, \dots, A_N$, where A_i denotes the weight of vertex i ($0 \leq A_i \leq 10^{18}$).

Each of the next $N - 1$ lines contains two integers x, y denoting the pair of vertices connected by an edge ($1 \leq x, y \leq N, x \neq y$). It is guaranteed that the given graph is a tree.

For all test cases, the sum of N is at most 1000.

Output

For each test case, output a binary string of length $K + 1$. The i -th character should be '1' if it is possible to achieve the desired goal by deleting exactly $i - 1$ edges. Otherwise, the i -th character should be '0'.

Example

1	3
2	4 3 1 2
3	1 1 1 1
4	1 2
5	2 3
6	3 4

```

7 | 4 3 1 2
8 | 1 1 1 1
9 | 1 2
10 | 1 3
11 | 1 4
12 | 4 3 0 0
13 | 1 1 1 1
14 | 1 2
15 | 1 3
16 | 1 4

```

```

1 | 0111
2 | 0011
3 | 0000

```

简体中文

给定一棵有 N 个顶点的树，顶点编号从 1 到 N 。这棵树是顶点带权的。换句话说，树的每个顶点都被赋予了一个非负整数权重。

我们将从树中删除一些边。删除之后，对于每个连通分量，顶点权重之和应该在范围 $[L, R]$ 内。

对于所有整数 $0 \leq i \leq K$ ，判断是否可以通过恰好删除 i 条边来实现这个目标。

输入

第一行包含一个整数 T ，表示测试用例的数量。接下来是 T 个测试用例，每个测试用例遵循以下格式：

每个测试用例的第一行包含四个整数 N, K, L , 和 R ($1 \leq N \leq 1000, 0 \leq K \leq \min(50, N - 1), 0 \leq L \leq R \leq 10^{18}$)。

下一行包含 N 个整数 $A_1, A_2, \dots, A_N$ ，其中 A_i 表示顶点 i 的权重 ($0 \leq A_i \leq 10^{18}$)。

接下来的 $N - 1$ 行，每行包含两个整数 x, y ，表示通过一条边连接的两个顶点 ($1 \leq x, y \leq N, x \neq y$)。保证给定的图是一棵树。

对于所有测试用例， N 的总和不超过 1000。

输出

对于每个测试用例，输出一个长度为 $K + 1$ 的二进制字符串。第 i 个字符应该是 '1'，如果可以通过恰好删除 $i - 1$ 条边来实现目标。否则，第 i 个字符应该是 '0'。

示例

1	3
2	4 3 1 2
3	1 1 1 1
4	1 2
5	2 3
6	3 4
7	4 3 1 2
8	1 1 1 1
9	1 2
10	1 3
11	1 4
12	4 3 0 0
13	1 1 1 1
14	1 2
15	1 3
16	1 4

1	0111
2	0011
3	0000

Problem D. Game

在发现编号从0到 $n - 1$ 的 n 颗行星后，法老们开始建造由**单向传送器**组成的行星交通系统。每个传送器都有起点行星和终点行星。当游客在起点行星使用传送器时，会被传送到终点行星。注意传送器的起点和终点可能是同一颗行星。一个起点为 u 、终点为 v 的传送器记作 (u, v) 。

为了鼓励广泛使用传送系统，法老们设计了一个游客在旅行途中可以参与的游戏。游客可以从任意行星开始游戏。行星 $0, 1, \dots, k - 1$ ($k \leq n$) 被称为**特殊行星**。每当游客进入特殊行星时，就会获得一枚印章。

当前，对于每个 i ($0 \leq i \leq k - 2$)，都有一个传送器 $(i, i + 1)$ 。这些 $k - 1$ 个传送器被称为**初始传送器**。

新的传送器会逐个添加。随着新传送器的加入，游客有可能获得无限数量的印章。具体来说，当存在满足以下条件的行星序列 $w[0], w[1], \dots, w[t]$ 时就会发生这种情况：

- $1 \leq t$
- $0 \leq w[0] \leq k - 1$
- $w[t] = w[0]$
- 对于每个 i ($0 \leq i \leq t - 1$)，都存在传送器 $(w[i], w[i + 1])$ 。

注意游客可以使用初始传送器和**任何**已添加的传送器。

你的任务是帮助法老们在每次添加传送器后，验证游客是否能获得无限数量的印章。

实现细节

你需要实现以下函数：

```
init(int n, int k)
```

- n ：行星数量。
- k ：特殊行星数量。
- 该函数仅被调用一次，在所有 `add_teleporter` 调用之前。

```
int add_teleporter(int u, int v)
```

- u 和 v ：新增传送器的起点和终点行星。
- 该函数最多被调用 m 次 (m 值参见约束条件)。
- 若添加传送器 (u, v) 后游客能获得无限印章，则返回1；否则返回0。
- 一旦该函数返回1，程序将立即终止。

示例

示例1

考虑以下调用：

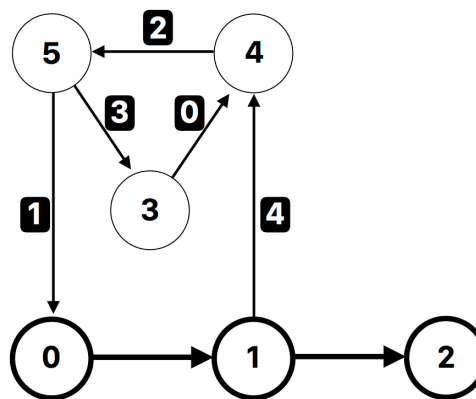
`init(6, 3)`

本例中有6颗行星和3颗特殊行星。行星0、1和2是特殊行星。初始传送器为(0,1)和(1,2)。

假设评测器依次调用：

- (0) `add_teleporter(3, 4)` : 应返回0
- (1) `add_teleporter(5, 0)` : 应返回0
- (2) `add_teleporter(4, 5)` : 应返回0
- (3) `add_teleporter(5, 3)` : 应返回0
- (4) `add_teleporter(1, 4)` : 此时游客可获得无限印章。例如游客从行星0出发，按1, 4, 5, 0, 1, 4, 5, 0, ...顺序访问行星。因此应返回1，程序终止。

下图展示了这个例子。特殊行星和初始传送器以粗体显示。通过 `add_teleporter` 添加的传送器按顺序标记为0至4。



示例2

考虑以下调用：

`init(4, 2)`

本例中有4颗行星和2颗特殊行星。行星0和1是特殊行星。初始传送器为(0,1)。

假设评测器调用：

- `add_teleporter(1, 1)` : 添加传送器(1,1)后，游客可获得无限印章。例如游客从行星1出发，通过传送器(1,1)无限次进入行星1。因此应返回1，程序终止。

附件包中提供其他样例输入/输出。

约束条件

- $1 \leq n \leq 300\,000$
- $1 \leq m \leq 500\,000$
- $1 \leq k \leq n$

对于每次 `add_teleporter` 调用：

- $0 \leq u \leq n - 1$ 和 $0 \leq v \leq n - 1$
- 在添加传送器 (u, v) 前，不存在从行星 u 到行星 v 的传送器。

子任务

1. (2分) $n = k, n \leq 100, m \leq 300$
2. (10分) $n \leq 100, m \leq 300$
3. (18分) $n \leq 1\,000, m \leq 5\,000$
4. (30分) $n \leq 30\,000, m \leq 50\,000, k \leq 1\,000$
5. (40分) 无额外限制

样例评测器

样例评测器按以下格式读取输入：

- 第1行： $n\ m\ k$
- 第 $2 + i$ 行 ($0 \leq i \leq m - 1$)： $u[i]\ v[i]$

样例评测器先调用 `init`，然后按顺序对 $u = u[i]$ 和 $v = v[i]$ 调用 $i = 0, 1, \dots, m - 1$ 。

输出首个返回1的 `add_teleporter` 调用索引（介于0和 $m - 1$ 之间），若所有 `add_teleporter` 调用都返回0则输出 m 。

若某次 `add_teleporter` 调用返回非0或1的整数，评测器将输出 -1 并立即终止程序。# 游戏

在发现编号从0到 $N - 1$ 的 N 颗行星后，法老们开始建造由**单向传送器**组成的行星交通系统。每个传送器都有起点行星和终点行星。当游客在起点行星使用传送器时，会被传送并进入终点行星。注意传送器的起点和终点可能是同一颗行星。一个起点为 u 、终点为 v 的传送器记作 (u, v) 。

为了鼓励广泛使用传送系统，法老们设计了一个游客在行星间旅行时可以参与的游戏。游客可以从任意行星开始游戏。行星 $0, 1, \dots, K - 1$ ($K \leq N$) 被称为**特殊行星**。每当游客进入特殊行星时，就会获得一枚印章。

当前，对于每个 i ($0 \leq i \leq K - 2$)，都有一个传送器 $(i, i + 1)$ 。这些 $K - 1$ 个传送器被称为**初始传送器**。

新的传送器会逐个添加。随着新传送器的加入，游客有可能获得无限数量的印章。具体来说，当存在满足以下条件的行星序列 $w_0, w_1, \dots, w_t$ 时就会发生这种情况：

- $1 \leq t$
- $0 \leq w_0 \leq K - 1$
- $w_t = w_0$
- 对于每个 i ($0 \leq i \leq t - 1$)，都存在传送器 (w_i, w_{i+1}) 。

注意游客可以使用初始传送器和**任何**已添加的传送器。

你的任务是帮助法老们在每次添加传送器后，验证游客是否能获得无限数量的印章。

实现细节

你需要实现以下过程和函数：

```
init(int N, int K)
```

- **N**：行星数量。
- **K**：特殊行星数量。
- 该过程仅被调用一次，在所有 `add_teleporter` 调用之前。

以下函数在每次添加传送器时被调用。

```
int add_teleporter(int u, int v)
```

- **u** 和 **v**：新增传送器的起点和终点行星。
- 该函数最多被调用 M 次（ M 值参见约束条件）。
- 若添加传送器 (u, v) 后游客能获得无限印章，则返回1；否则返回0。
- 一旦该函数返回1，程序将立即终止。

示例

示例1

设 $N = 6$ 和 $K = 3$ 。

评测器首先调用 `init(6, 3)`。

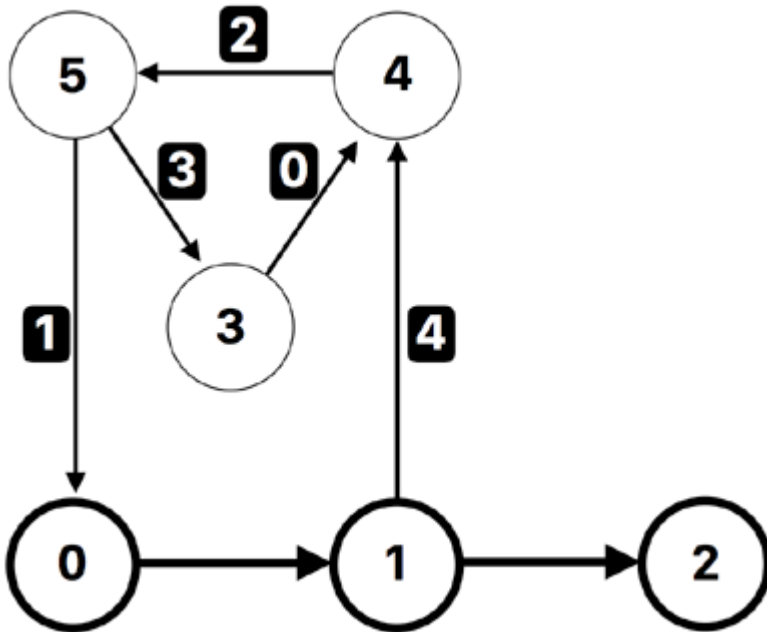
本例中有6颗行星和3颗特殊行星。行星0、1和2是特殊行星。初始传送器为 $(0, 1)$ 和 $(1, 2)$ 。

假设评测器依次调用：

- (0) `add_teleporter(3, 4)`：应返回0
- (1) `add_teleporter(5, 0)`：应返回0

- (2) `add_teleporter(4, 5)` : 应返回0
- (3) `add_teleporter(5, 3)` : 应返回0
- (4) `add_teleporter(1, 4)` : 此时游客可获得无限印章。例如游客从行星0出发，按1, 4, 5, 0, 1, 4, 5, 0, ...顺序访问行星。因此应返回1，程序终止。

下图展示了这个例子。特殊行星和初始传送器以粗体显示。通过 `add_teleporter` 添加的传送器按顺序标记为0至4。



压缩包中的文件 `sample-01-in.txt` 对应此示例。

示例2

设 $N = 4$ 和 $K = 2$ 。

评测器首先调用 `init(4, 2)`。

本例中有4颗行星和2颗特殊行星。行星0和1是特殊行星。初始传送器为(0, 1)。

假设评测器调用：

- `add_teleporter(1, 1)` : 添加传送器(1, 1)后，游客可获得无限印章。例如游客从行星1出发，通过传送器(1, 1)无限次进入行星1。因此应返回1，程序终止。

压缩包中的文件 `sample-02-in.txt` 对应此示例。

附件包中还提供其他样例输入/输出。

约束条件

- $1 \leq N \leq 300\,000$

- $1 \leq M \leq 500\,000$
- $1 \leq K \leq N$
- 对任何 `add_teleporter` 调用, $0 \leq u \leq N - 1$ 和 $0 \leq v \leq N - 1$ 。对任何 `add_teleporter` 调用, 在添加传送器 (u, v) 前, 不存在从房间 u 到行星 v 的传送器。

子任务

1. (2分) $N = K, N \leq 100, M \leq 300$
2. (10分) $N \leq 100, M \leq 300$
3. (18分) $N \leq 1\,000, M \leq 5\,000$
4. (30分) $N \leq 30\,000, M \leq 50\,000, K \leq 1\,000$
5. (40分) 无额外限制

样例评测器

样例评测器按以下格式读取输入：

- 第1行： $N\ M\ K$
- 第 $2 + i$ 行 ($0 \leq i \leq M - 1$)： $u_i\ v_i$

样例评测器先调用 `init`, 然后按顺序对 $u = u_i$ 和 $v = v_i$ 调用 $i = 0, 1, \dots, M - 1$ 。

输出首个返回1的 `add_teleporter` 调用索引 (介于0和 $M - 1$ 之间), 若所有 `add_teleporter` 调用都返回0则输出 M 。

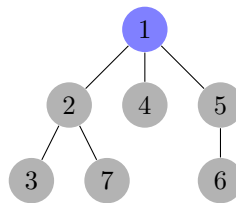
若某次 `add_teleporter` 调用返回非0或1的整数, 评测器将输出 -1 并立即终止程序。

DFS Order 4

Input file: **standard input**
Output file: **standard output**
Time limit: 2 seconds
Memory limit: 1024 megabytes

Little Cyan Fish, also known as Qingyu Xiao, loves the concept of DFS order. Today, he has a rooted tree T with n vertices labeled from 1 to n . The root of the tree is vertex 1, and the parent of vertex i ($2 \leq i \leq n$) is vertex f_i ($1 \leq f_i < i$).

A DFS order $D = (D_1, D_2, \dots, D_n)$ represents the sequence of nodes visited during a depth-first search of the tree. A vertex appearing at the j -th position in this order (where $1 \leq j \leq n$) indicates that it is visited after $j - 1$ other vertices. During the depth-first search, if a vertex has multiple children, they are visited in **ascending order** of their indices. Thus, in this problem, each rooted tree has a **unique** DFS order.



A tree with 7 vertices. The DFS Order of the tree is $[1, 2, 3, 7, 4, 5, 6]$.

The following pseudocode describes a way to generate the DFS order given a rooted tree T . T is uniquely represented by the array $f = \{f_2, \dots, f_n\}$. The function `GENERATE()` returns the DFS order starting at the root vertex 1:

Algorithm 1 An implementation of the depth-first search algorithm

```
1: procedure DFS(vertex  $x$ )
2:   Append  $x$  to the end of dfs_order
3:   for each child  $y$  of  $x$  do                                 $\triangleright$  Children are iterated in ascending order of index.
4:     DFS( $y$ )
5:   end for
6: end procedure
7: procedure GENERATE()
8:   Let dfs_order be a global variable
9:   dfs_order  $\leftarrow$  empty list
10:  DFS(1)
11:  return dfs_order
12: end procedure
```

Let D be the array returned by `GENERATE()`. There are $(n - 1)!$ different possible configurations for the array f , each representing a distinct tree T . Little Cyan Fish wonders: for all these $(n - 1)!$ configurations of f , how many distinct DFS orders D can be generated? We consider two DFS orders D and D' to be different if and only if there exists an index $1 \leq i \leq n$ such that $D_i \neq D'_i$. Given that the number can be very large, your task is to compute this number modulo a given prime integer P .

Input

The first line of the input contains two integers n and P ($1 \leq n \leq 800$, $10^8 \leq P \leq 1.01 \times 10^9$).

It is guaranteed that P is a prime number.

Output

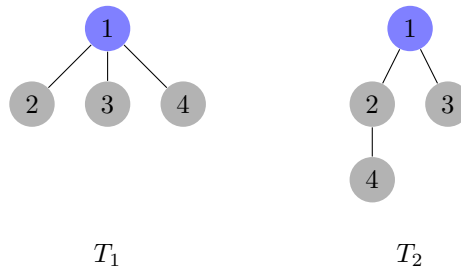
Output a single line containing a single integer, indicating the answer.

Examples

standard input	standard output
4 114514199	2
10 998244353	11033
100 1000000007	270904395

Note

In the first example, there are two distinct DFS orders: $D_1 = [1, 2, 3, 4]$ and $D_2 = [1, 2, 4, 3]$, which can be obtained by $T_1 : f_2 = 1, f_3 = 1, f_4 = 1$ and $T_2 : f_2 = 1, f_3 = 1, f_4 = 2$, respectively.



Problem F. 小火车

Time Limit 2000 ms

Mem Limit 524288 kB

hehe 蚤在下山市发现了当地特色的玩具小火车！

hehe 蚤立马迫不及待的开始玩了起来。小火车运行在一个长度为 p 的圆形铁轨上。小火车上有 n 个拨杆，把第 i 个拨杆向左拨可以让小火车向前开 a_i 单位距离，而向右拨可以让小火车向后开相同距离。每个拨杆最多被拨一次。

hehe 蚤想让小火车开回原位，他想让你告诉他，该怎么拨拨杆才能让小火车运行后回到原位。你不必拨每一个拨杆，但不能一个拨杆都不拨，否则就不好玩了。

显然拨杆的顺序并不会影响最后火车的位置，所以你只需要输出每个拨杆的拨法。

一句话题意：给定 n, p 和长度为 n 的整数序列 a ，你需要输出整数序列 b 满足：

1. $b_i \in \{-1, 0, 1\}$ 且不全为 0。

2. $\sum a_i b_i \equiv 0 \pmod{p}$

保证 $2^n > p$ 。

输入格式

第一行两个整数 n, p 表示集合大小和模数。

第二行 n 个整数 a_i 表示集合内容。

输出格式

若无解输出 **IMPOSSIBLE**。

否则输出一行 n 个 -1 到 1 的整数，第 i 个整数为 -1 表示第 i 个拨杆向左拨， 1 表示第 i 个拨杆向右拨， 0 表示第 i 个拨杆将不会被操作。

如有多组解输出任意一组均可。

样例一

input

3 7

1 2 3

output

1 1 -1

explanation

小火车会向后开 $1 + 2 = 3$ 单位距离，向前开 3 单位距离，回到原位。

样例二

见附件下载中的 `ex_train2.in` 和 `ex_train2.ans`。

数据范围

子任务编号	$n \leq$	特殊性质	分值
1	15	无	20
2	20	无	20
3	36	数据随机	20
4	40	无	40

数据随机是指： a_i 在 $[0, p)$ 范围内独立均匀随机。

第 3 个子任务有 5 组测试数据。

对于所有数据，保证 $1 \leq p < 2^n, 1 \leq n \leq 40, 0 \leq a_i < p$ 。

时间限制：2s

空间限制：512MB

Problem G. 喂鸽子

Time Limit 1000 ms

Mem Limit 262144 kB

小Z是养鸽子的人。一天，小Z给鸽子们喂玉米吃。一共有 n 只鸽子，小Z每秒会等概率选择一只鸽子并给他一粒玉米。一只鸽子饱了当且仅当它吃了的玉米粒数量 $\geq k$ 。小Z想要你告诉他，期望多少秒之后所有的鸽子都饱了。

假设答案的最简分数形式为 $\frac{a}{b}$ ，你需要求出 w ，满足 $a \equiv b \cdot w \pmod{998244353}$ ($0 \leq w < 998244353$)。

ps:为了方便选手确认自己程序的正确性，下发文件中包含一个模拟器的源码，请选手自行编译。该模拟器会无限地重复模拟题目中描述的过程，并将每次模拟的秒数的平均值以**实数**的形式输出，该模拟器能够保证3 ~ 4位有效数字的精确度。

输入格式

一行两个正整数 n, k 。

输出格式

一行一个数表示答案。

样例一

input

1 1

output

1

限制与约定

本题采用子任务的方式评测。

子任务一 (4pts) : $n = 1$ 。

子任务二 (11pts) : $k = 1$ 。

子任务三 (19pts) : $k = 2$ 。

子任务四 (46pts) : $k \leq 50$ 。

子任务五 (20pts) : 无特殊限制。

对于所有的数据, $n \leq 50$, $k \leq 1000$ 。

时间限制 : 1s

空间限制 : 256MB

下载

[样例数据及模拟器下载](#)

Exchanging Kubic

Input file: **standard input**
Output file: **standard output**
Time limit: 1 second
Memory limit: 1024 megabytes

This is an interactive problem.

In computer science, the maximum sum subarray problem, also known as the maximum segment sum problem, is the task of finding a contiguous subarray with the largest sum, within a given one-dimensional array $A_1, A_2, \dots, A_n$ of numbers. Formally, the task is to find indices i and j , such that the following sum is as large as possible:

$$\sum_{i \leq k \leq j} A_k$$

It is also possible to choose an empty subarray, which means you found an empty array with the sum 0. The value of the maximum sum subarray is denoted by $MSS(A)$. For example, $MSS([-2, 1, 4, -3, 5]) = 7$, $MSS([-5]) = 0$, and $MSS([-1, -2]) = 0$.

Little Cyan Fish is taking a course on Kubic Theory at Powerful Kubic University (PKU). Today, Prof. Kubic asked Little Cyan Fish to play the following game with him during the course:

Prof. Kubic has a sequence of integers $a_1, a_2, \dots, a_n$. Little Cyan Fish could ask Prof. Kubic at most $2n$ questions in the following form:

- `? l r`: query the value of $MSS(a_l, a_{l+1}, \dots, a_r)$.

The task of Little Cyan Fish is to report a sequence $b_1, b_2, \dots, b_n$ satisfying:

- $MSS(a_l, a_{l+1}, \dots, a_r) = MSS(b_l, b_{l+1}, \dots, b_r)$ for all $1 \leq l \leq r \leq n$.

Little Cyan Fish found this task is very challenging. Can you help him to develop a strategy to finish Prof. Kubic's task?

Interaction Protocol

There are multiple test cases in a single test file. The first line of the input contains a single integer T ($1 \leq T \leq 10^4$), indicating the number of the test cases.

For each test case, the first line of the input contains a single integer n ($1 \leq n \leq 2000$).

Then, the interaction begins. You may perform at most $2n$ queries in each test case. To perform a query, you need to print a single line `"? l r"` ($1 \leq l \leq r \leq n$), indicating a query. Then, you need to read the result of your query from the standard input.

To give your answer, you need to print `"! b_1 b_2 \dots b_n"`. You need to ensure that $-10^{15} \leq b_i \leq 10^{15}$. Printing the answer is not considered a query and does not count toward the $2n$ limit. After printing you answer, you need to read the next case, or terminate your program immediately.

After printing a query, do **NOT** forget to output end of line and flush the output. To do this, use `"fflush(stdout)"` or `"cout.flush()"` in C++, `"System.out.flush()"` in Java, `"flush(output)"` in Pascal, or `"stdout.flush()"` in Python.

It is guaranteed that $-10^9 \leq a_i \leq 10^9$, and the sum of n over all test cases does not exceed 10^4 .

In this problem, it is guaranteed that the interactor is **non-adaptive**. That is, the values of a_i are decided before the interaction process. They will not be changed based on your queries.

Example

standard input	standard output
2	
3	? 1 1
1	? 2 2
0	? 3 3
1	! 1 -1 1
5	? 1 3
4	? 3 5
5	! 2 -1 3 -4 5